OpenClaw Color Line Patrol

OpenClaw Color Line Patrol

1. Course Content
    Learning Objectives
2. Preparation
3. Case Demonstration
4. Source Code Analysis
5. Color Calibration

1. Course Content

Learning Objectives

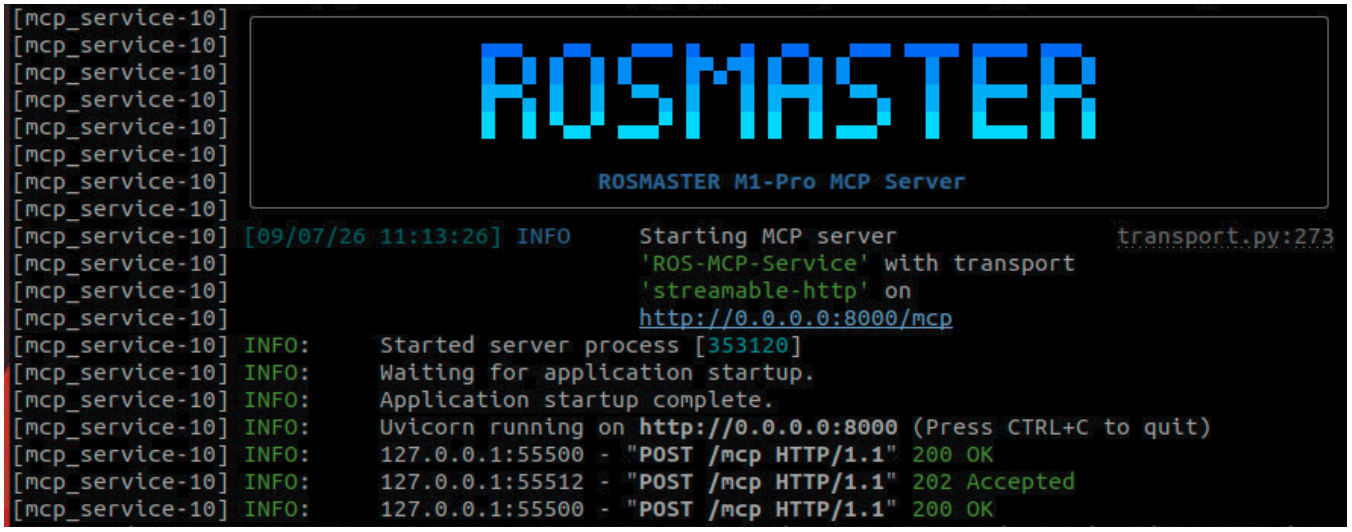
Through the study of this chapter, you will master the following skills:

- **Master the `FollowLine` tool invocation**: Learn to use the MCP tool to start an automatic line-following task for a specified color (supporting four colors: red, green, blue, yellow).
- **Possess color calibration capability**: Be able to use the `colorSelect` tool to adjust HSV parameters according to the actual ambient lighting, to optimize recognition robustness.

2. Preparation

- Demonstration props: red tape
- Lay a colored tape on the ground (supports: red, green, blue, yellow)

```
ros2 launch yahboomcar_bringup mcp_service.launch.py use_camera:=true
```

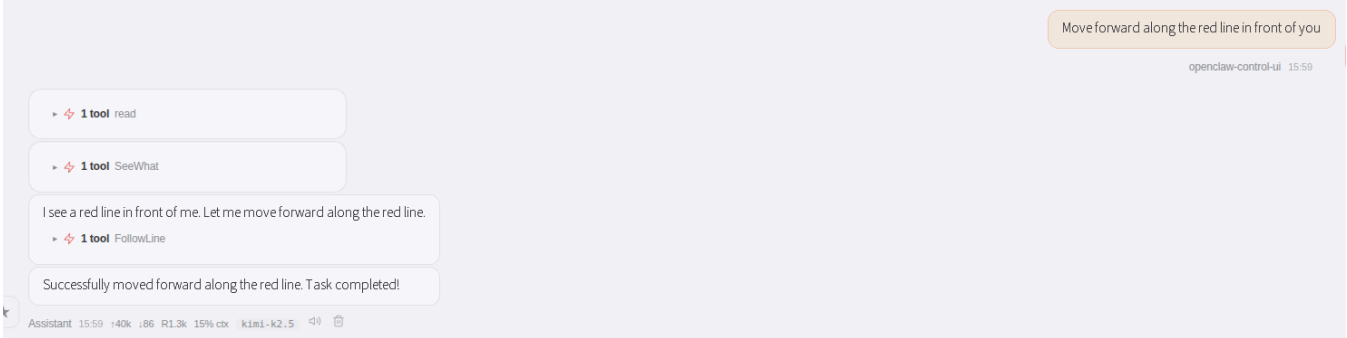


3. Case Demonstration

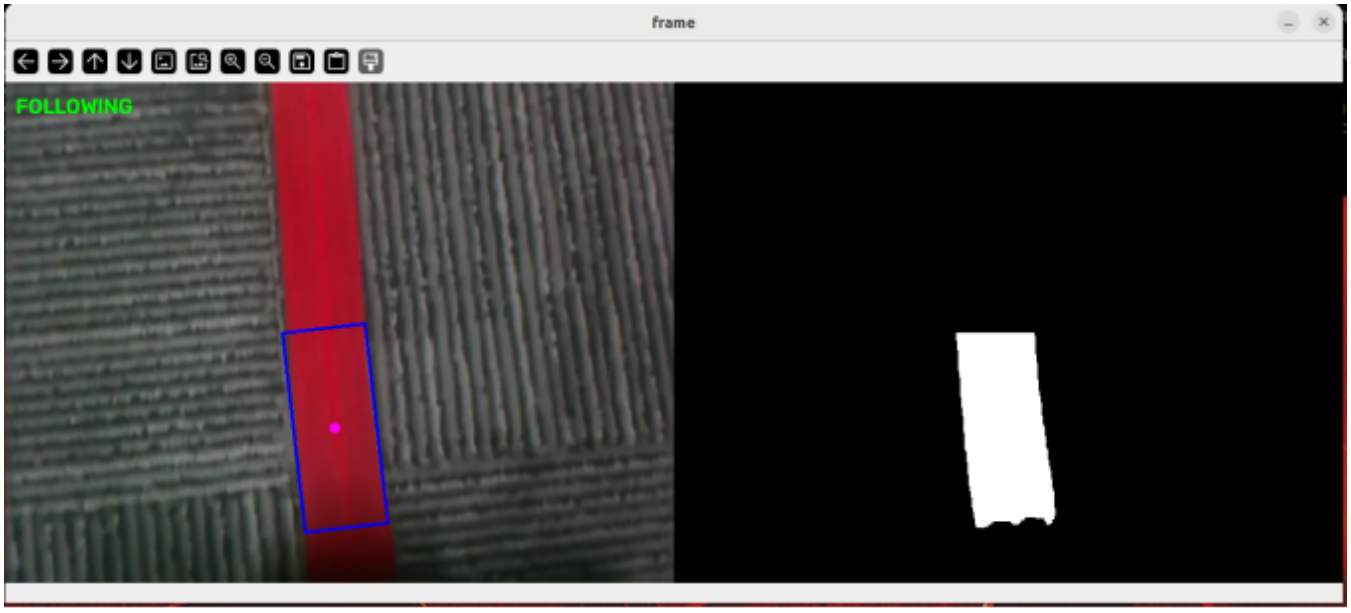
> **Tip**
>
> - You can write the command yourself according to your needs. The following is a case

> demonstration. Just choose any one of the interaction methods. The demonstration here uses the web-based WebChat as an example.

Command	Parameter	Function
FollowLine	`--color` (line-following color; options: red, green, blue, yellow)	Follow a colored line on the ground



- When the `FollowLine` tool is called, the system pops up a display window showing the original RGB image and the binarized image in real time, making it easy to observe the recognition effect.



4. Source Code Analysis

- Source code path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains_pre/multi_brains_pre/action_controller.py
```

- `_start_follow_line_clear(color)`: Start an independent line-following subprocess.
- `_stop_follow_line_clear()`: First stop the existing line-following process to avoid duplicate startup.
- `file_manager.GetFileAndCheckExist("config_file")`: Load the common parameter configuration

file to ensure the line-following node can read the correct HSV threshold parameters.

- `-p color:={color}`: Specify the target color through a command-line parameter, supporting four colors: red/green/blue/yellow.
- `-p mode:=integrated`: Set the integrated mode, which automatically clears the AprilTag obstacles on the path while following the line.
- `subprocess.Popen(cmd)`: Start the `follow_line_clear` node as an independent subprocess, without blocking the main program's execution.
- `_wait_follow_line_exit()`: Poll and wait for the line-following subprocess to exit, checking the process status every 0.5 seconds.
- `kill_process_tree(pid)`: Terminate the line-following process and all of its subprocesses, ensuring resources are fully released.
- `FollowLine(color)`: The main line-following function, completing the color line patrol task in a three-step process.
- `_pubSix_Arm(self.waste_detect_pose)`: Move the robotic arm to the preset observation pose to ensure the camera's field of view can cover the road ahead.
- `finally: _stop_follow_line_clear()` + `InitArmPose()`: Regardless of whether the line following succeeds or fails, the subprocess is ultimately terminated and the robotic arm is restored to its initial pose.

```
    def _start_follow_line_clear(self, color):
        """Launch the standalone follow_line_clear process in integrated mode."""
        self._stop_follow_line_clear()
        cmd = ['ros2', 'run', 'common_demo', 'follow_line_clear', '--ros-args']
        try:
            common_param_file = file_manager.GetFileAndCheckExist("config_file")
            cmd += ['--params-file', common_param_file]
        except Exception as e:
            self.get_logger().warn(f"Failed to load common param file: {e}")
        cmd += ['-p', f'color:={color}', '-p', 'mode:=integrated']
        self.get_logger().info(f"Starting follow_line_clear process: {' '.join(cmd)}")
        self.follow_line_process = subprocess.Popen(cmd)

    def _stop_follow_line_clear(self):
        if self.follow_line_process is not None:
            if self.follow_line_process.poll() is None:
                kill_process_tree(self.follow_line_process.pid)
            self.follow_line_process = None

    def _wait_follow_line_exit(self):
        """Wait until the follow_line_clear process exits by itself, return its exit
code."""
        while rclpy.ok():
            if self.follow_line_process is None:
                return None
            ret = self.follow_line_process.poll()
            if ret is not None:
                return ret
            time.sleep(0.5)
```

```
        return None

    def FollowLine(self,color:str) -> list[bool,str]:
        '''Follow the line of the given color and remove apriltag obstacles on the way,
finish when the line ends'''
        self._pubSix_Arm(self.waste_detect_pose)

        color = str(color).strip().strip("'\"")
        if color not in ['red', 'green', 'blue', 'yellow']:
            error_msg = f"FollowLine: invalid color '{color}', must be
red/green/blue/yellow"
            self.get_logger().error(Fore.RED+error_msg+Fore.RESET)
            return [False, error_msg]

        try:
            self.get_logger().info(f"FollowLine: start following '{color}' line")
            self._start_follow_line_clear(color)
            exit_code = self._wait_follow_line_exit()
            if exit_code is None:
                return [False, "FollowLine: process not running or rclpy shutdown"]
            if exit_code == 0:
                self.get_logger().info(f"FollowLine: '{color}' line followed and cleared")
                return [True, f"FollowLine finished: '{color}' line followed and cleared"]
            self.get_logger().warn(f"FollowLine process exited with code {exit_code}")
            return [False, f"FollowLine failed: process exited with code {exit_code}"]
        finally:
            self._stop_follow_line_clear()
            self.InitArmPose()
```

Core Logic Description:

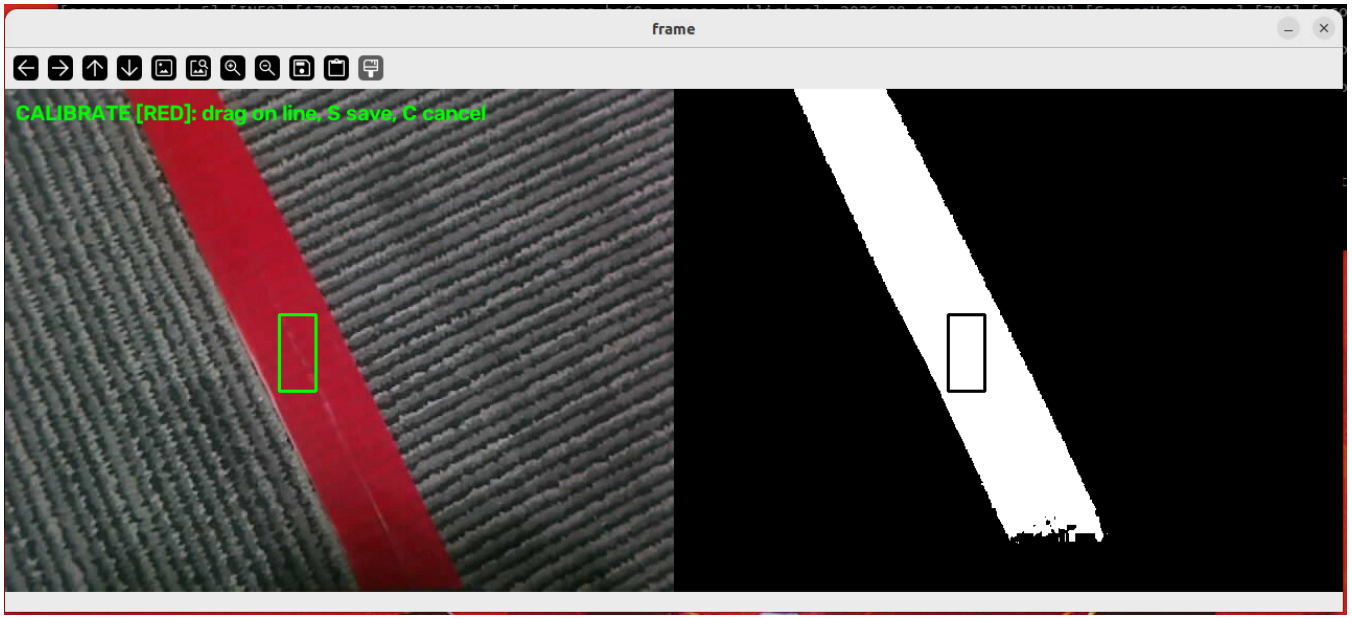
1. **Pose preparation**: First call `_pubSix_Arm` to move the robotic arm to the preset observation position, ensuring the camera's field of view can cover the road ahead.
2. **Parameter mapping**: Convert the color string input by the user (such as "red") into the numeric ID agreed upon by the underlying algorithm, for HSV threshold matching.
3. **Process management**: Use `subprocess.Popen` to independently start the `follow_line_clear` node. This design allows the line-following algorithm to run at high frequency in an independent process, without affecting the response speed of the main controller.
4. **Synchronization and cleanup**: Achieve synchronization between the main program and the subprocess through a `Future` object. When the line-following task ends (such as when the endpoint is detected or manually terminated), the subprocess is automatically killed and the robotic arm is restored to its initial state.

5. Color Calibration

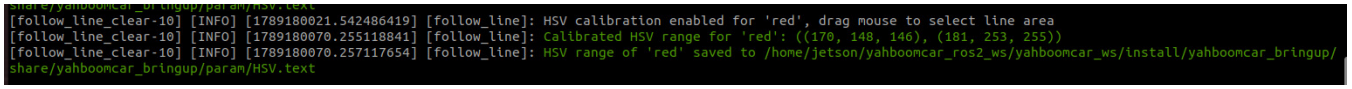
- Refer to the calibration method in the preceding robotic arm chapter lesson.

```
ros2 launch yahboomcar_bringup arm_demo.launch.py demo:=follow_line_clear
```

- First box out the target color. Here, red is used as an example. Check whether the binarized image recognition is complete, as shown in the figure below.



- Press the s key in the frame window to save the current color HSV values to the parameter file. The terminal will show a corresponding prompt. Then `ctrl+c` to close the program.



- HSV parameter file path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/HSV.text
```